

CSC 665.04

Dr. Sara El Alaoui

Homework 2

Group 5 Members: Ammar Almeher, Davis Qin

Problem I:**Part 1. b:**

- We use the MCTSNode class to represent each node in the search tree. Each node stores the current state, its parent, the action from its parent, its children, and the untried actions. We use N to track how many times the node has been visited and W to track its total reward (win = +1, tie = 0, loss = -1). These values are updated during backpropagation and are used to calculate the UCT score during selection.
- We store an untried_actions list in each node to keep track of all available actions that have not been tried. During expansion, we use pop() to take one action from the list and use succ() to perform that action. Then, we create a new child node for the resulting state. This helps us explore actions that have not been tried yet.
- We use random.choice() to randomly select one legal action from all available actions. We continue selecting and performing random actions until the game reaches a terminal state. Then, we calculate the reward based on the final result.

Part 2:

=== Reward mode: winloss ===								
Budget	Games	Win%	Loss%	Tie%	AvgDiff	MCTS ms/mv	MM ms/mv	
20	60	85.0%	11.7%	3.3%	17.60	0.12	306.94	
200	60	88.3%	11.7%	0.0%	19.50	1.08	309.46	
2000	60	90.0%	10.0%	0.0%	21.77	10.48	305.70	
20000	60	95.0%	3.3%	1.7%	21.57	88.88	284.37	

Part 3:

1. From these four groups of results, we can see that as the MCTS budget increases, the computation time for each move also increases significantly. At the same time, the MCTS win rate increases from 85% to 95%. The average score difference is positive for all four budgets, which means that MCTS scores higher than minimax on average. We can also observe that MCTS has a lower computation time per move than minimax in all four experiments.

2. Answer:

- a. I do not think the high MCTS win rate is logical. MCTS uses simulations to estimate the best action based on the paths it has explored, while minimax searches all possible actions and chooses the optimal action. Therefore, MCTS should not normally have such a high win rate against minimax if both algorithms are implemented with the same objective.
- b. I think Minimax has a low win rate because the level at which the game reaches a terminal state can affect the evaluation and the final action. For example, suppose the player's final score is 30 points, and the AI's is 50 points. If `is_maximizing` is `True` when the game ends, the code calculates `aiScore - playerScore`, which gives +20. However, if `is_maximizing` is `False`, it calculates `playerScore - aiScore`, which gives -20. Therefore, the same winning result for the AI can have different values depending on when the game reaches the terminal state. This can cause Minimax to incorrectly evaluate some results and choose an action that is not actually optimal.
- c. In `benchmark.py`, Minimax always plays as the AI. In line 67, `is_maximizing=True: _, action = cl.minimax(state, is_maximizing=True)`. So I changed the terminal evaluation to always evaluate the result from the AI's perspective.

Previous:

`return (p2 - p1 if is_maximizing else p1 - p2), None`

Changed to:

`return p2 - p1, None`

The result below:

=== Reward mode: winloss ===									
Budget	Games	Win%	Loss%	Tie%	AvgDiff	MCTS	ms/mv	MM	ms/mv
20	60	25.0%	75.0%	0.0%	-9.33	0.12	366.37		
200	60	33.3%	61.7%	5.0%	-7.03	1.05	373.17		
2000	60	28.3%	66.7%	5.0%	-6.67	9.24	375.62		
20000	60	28.3%	68.3%	3.3%	-7.37	95.40	393.78		

3. One reason MCTS may still struggle is that we use `random.choice()` to select actions during the rollout. This means that the simulation results depend on random choices and may not always find the optimal actions. As the budget increases, MCTS has more simulations, and the win rate increases from 85% to 95% in my results. However, even with a large budget, the random rollouts cannot guarantee that MCTS will choose the optimal action every time.

Problem II:

Part 1:

a. $(\neg P \wedge Q) \vee (P \wedge \neg Q)$

Part 2:

```
PS C:\Users\Devis\OneDrive\Documents\SFSU\Fall2026\CSC665-A  
I\HW\CSC-665-group-hw\HW2\homework2_code\knights> python ./  
puzzle.py  
Puzzle 1  
    A is a Knave  
Puzzle 2  
    A is a Knave  
    B is a Knight  
Puzzle 3  
    A is a Knight  
    B is a Knave  
    C is a Knight  
PS C:\Users\Devis\OneDrive\Documents\SFSU\Fall2026\CSC665-A  
I\HW\CSC-665-group-hw\HW2\homework2_code\knights>
```

GenAI Usage: I used GenAI to help me understand the provided MCTS and logic code, debug Python implementation issues, and proofread my written responses. I also asked high-level questions about evaluating equivalent performance. I reviewed and edited the suggestions and made sure I understood the code and concepts used in my submission.